AI at Yalla

Playbook, AI review, and data & recommendations plan

Author: Youssef Tekriti · November 2026 · branch claude/ai-architecture-review-a430a4

1. Overview
2. Building AI features: the playbook
3. Yalla AI review
4. Data and recommendations

AI at Yalla

How we build AI features, and the state of the AI in this repo.

Doc	Read it when
Playbook	You're designing any AI feature: code vs search vs LLM vs agent vs ML, which model, evals, guardrails, checklist
Yalla AI review	You want to know how AI works in Yalla today, what's wrong with it, and the target architecture
Data and recommendations	You're adding logging, or asking "should we build a recommendation engine?"

The one-line version: use an LLM for language and taste, code for rules and maths, search and ranking for picking from lists, and trained ML only once you have the data. Start logging the data now.

Building AI features: the playbook

> How we decide **what kind of intelligence** a feature needs, and how to build it so it is fast, cheap, testable and gets better over time. Examples come from Yalla, but the rules apply to any AI product.

Read next: Yalla AI review (this playbook applied to our code) · Data and recommendations (what to log, and when a recommendation engine makes sense)

---

1. Start from the job, not the model

"Add AI" is not a design. Before writing a prompt, split the feature into **jobs** and ask, for each one, *what is the cheapest thing that does this job well?*

Our PRD already does this for Yalla (jobs A to F in PRD §5.2):

Job	What it really is	Best tool
A. Capture intent	A form	Code
B. Understand intent	Messy text in 3 scripts → structured fields	**LLM** (small model)
C. Fetch places	Search over a database	**API** (Google Places)
D. Compose plans	Pick stops that fit hours, time and driving limits	**Search algorithm** in code, with ranking
E. Write copy	Short, warm, local wording	**LLM**
F. Save and share	Storage	Code

The mistake to avoid is handing several jobs to one big prompt because it "works in the demo". You lose speed, cost control, testability and the ability to improve each job on its own.

---

2. The five tools, and when each one wins

```
flowchart TD
  Q[A job in your feature] --> R{Can you write the rule down<br/>and is it always
right?}
  R -->|yes| CODE[1. Plain code / rules]
  R -->|no| D{Is the input free-form<br/>language, images or audio?}
  D -->|yes| L{Does it need several steps<br/>with tools you can't fix in advance?}
  L -->|no| LLM[3. LLM: single call or fixed workflow]
  L -->|yes| AG[4. LLM agent with tools]
  D -->|no| S{Is it picking or ordering items<br/>from a known set?}
  S -->|yes| H{Do you have interaction data?<br/>views, clicks, swaps, shares}
  H -->|not yet| HEU[2. Scoring formula + search in code<br/>+ start logging]
  H -->|yes, plenty| ML[5. Trained ranking / recommendation model]
  S -->|no| P{Predicting a number or label<br/>from structured data?}
  P -->|yes| ML
  P -->|no| LLM
```

#	Tool	Wins when	Loses when	Yalla example
1	**Code / rules**	The rule is known and must always hold: hours, budgets, time maths, geography	The input is fuzzy language	"Never schedule a stop when it's closed"
2	**Scoring + search** (no training)	Picking or ordering from a list, with no data yet	You have data and a fixed formula leaves quality on the table	Rating adjusted for review count, + distance, + profile match → best valid 3-stop route
3	**LLM, single call or fixed workflow**	Understanding or generating language; taste; one-off extraction	Exact facts, arithmetic, hard constraints, high volume at low latency	"بدنا نتغدى بالبترون" → `{area: Batroun, slots: [Lunch]}`
4	**LLM agent** (model chooses tools in a loop)	The steps can't be known in advance, mistakes are recoverable, and the value justifies the cost and latency	The steps are known. Then a fixed workflow is cheaper, faster and testable	Probably never in Yalla's request path
5	**Trained ML** (ranking, recommendations, classifiers)	You have many labelled examples or interactions and the pattern is statistical	Cold start, no data, or the rule is known	"People who loved this brunch spot also loved…" (later)

Rules of thumb

- **LLM proposes, code decides.** Let the model understand and suggest; let code check every hard fact (does it exist, is it open, does it fit the time) before a user sees it.

- **Never ask an LLM to do something you'll overwrite.** If code recomputes the times anyway, don't pay the model to compute them.
- **Workflow before agent.** If you can draw the steps as a fixed diagram, write them as code that calls the model at fixed points. Only reach for an agent loop when the path truly depends on what the model finds.
- **ML needs data; LLMs need examples.** A recommendation model with no interaction data is guesswork. An LLM with no eval set is also guesswork.

---

3. What LLMs are good and bad at

Good at	Bad at
Understanding messy language, typos, dialects, mixed scripts (Arabic, Franco-Arabic, English)	Arithmetic, time maths, distances
Pulling structured fields out of text	Following 15 hard rules at once, every time
Writing short copy with a tone	Knowing which places exist or are open *today*
Judging "taste" between a few good options	Picking from 150 candidates consistently
Summarising, classifying, translating	Being cheap and fast at high volume with a big model

The pattern that follows: **send the LLM small, focused tasks where language or taste matters, with the facts already checked by code.**

---

4. LLM harness patterns, simplest first

Pick the lowest rung that works. Each rung up adds cost, latency and things that can go wrong.

1. **Single call with structured output.** One prompt in, JSON checked against a schema out. Covers about 80% of product AI: parsing, extraction, classification, copy. *Yalla: parse the request; read a reel caption.*
2. **Fixed workflow (chain).** Code calls the model at set points with code steps in between: parse → retrieve → rank → write. *Yalla: the target architecture in the review.*
3. **Router.** A cheap call decides which path to take (for example, "is this a trip, a day out, or out of scope?").
4. **Tool-use agent.** The model decides which tools to call, in a loop, until done. Use only when all four hold: - **Complexity:** the steps can't be specified in advance. - **Value:** the outcome justifies the extra cost and latency. - **Viability:** the model is actually good at this kind of task. - **Recoverable errors:** mistakes can be caught and fixed (tests, review, undo).

A consumer "plan my Saturday" screen with a 5-second goal fails the latency test. Keep agents for back-office work, such as curating a new city's starter list overnight.

Every LLM call should have

- **A schema for the output** (Zod + the API's structured output), and code checks *after* it for anything that matters.
- **A typed failure path**: out of credits, busy, refused, cut off, didn't match the schema. Each one maps to a clear message for the user, never a blank screen. `askJSON` in `src/lib/ai.server.ts` already does this well.
- **A time budget.** Decide the worst-case latency up front, counting retries.
- **A log line**: route, model, effort, prompt version, input/output tokens, cache hits, latency, stop reason. Without it you can't evaluate or reduce cost (see data doc).
- **An eval set** before it ships (see §6).

---

5. Choosing a model

The Claude lineup

Prices are per million tokens, as of November 2026. Check the Anthropic pricing page before budgeting.

Model	ID	Input	Output	Positioning	Use it for
Haiku 5.5	`claude-haiku-5-5`	$0.10	$0.50	Fastest, cheapest	Parsing, extraction, classification, routing, high-volume steps on the user's critical path
Sonnet 5.5	`claude-sonnet-5-5`	$2	$10	Balanced	Copy with tone, re-ranking a few options for taste, most product features
Opus 5.5	`claude-opus-5-5`	$4	$20	Strongest everyday model	Hard reasoning, LLM-as-judge in evals, generating test data, offline curation
Fable 5.1	`claude-fable-5-1`	$10	$50	Most capable, long turns	Long agentic jobs and research. Not for a consumer request path

Haiku 5.5's price applies up to 100K-token prompts.

How to choose

1. **Start with the model the eval says is good enough, not the one that feels safest.** Run the same eval set on Haiku, Sonnet and Opus (at low effort too). Keep the cheapest one that passes the hard checks.
2. **Effort is a dial within one model.** `output_config.effort` (`low` → `max`) trades depth for tokens and latency. Opus 5.5 defaults to `medium`. Always try a bigger model at **low**

effort next to a smaller model before building a multi-model cascade. One model is simpler, and caches don't carry across models.
3. **Judge cost per finished task, not per request.** A cheap call that fails half the time and needs a retry isn't cheap.
4. **Offline work gets the Batch API**: about 50% cheaper and asynchronous. Use it for eval runs, LLM-as-judge grading and bulk enrichment such as writing descriptions for every place.
5. **Prompt caching** makes repeated *prefixes* about 10× cheaper to read. Put stable content first (system prompt, few-shot examples, a city's place list) and anything that changes per request last. It only pays off when the stable prefix is large (roughly 0.5K–4K tokens minimum, depending on the model) and reused within minutes.
6. **Plan for refusals and outages.** On Opus, Sonnet and Fable, enable the server-side `fallbacks` option. Haiku has no server-side fallback, so handle `stop_reason: "refusal"` in code.

Yalla's routes

Route	Today	Recommended starting point	Why
Parse request (job B)	Keywords in code, plus the planner prompt	**Haiku 5.5**, low effort, structured output; move up to Sonnet only if the Arabic/Franco eval fails	Short input, short output, on the critical path
Read reel caption	Opus 5.5, low effort	**Haiku 5.5**	Simple name extraction
Build the plan (job D)	Opus 5.5, medium effort, ~150 candidates	**Code** (scoring + search); optional Sonnet re-rank of the top ~8	Hard rules belong in code; the model only adds taste
Write copy (job E)	Inside the planner call	**Sonnet 5.5** (or Haiku if it passes the tone check)	Tone matters; input is just 3 fixed plans
Grade evals (LLM-as-judge)	None	**Opus 5.5** via the Batch API	Offline, so accuracy matters more than cost
Status ping	Opus	Haiku, or no LLM call at all	It only needs to prove the key works

---

6. Evals: how you know it works

No AI feature ships without an eval. Use three layers:

1. **Golden set (offline, in CI).** 30–100 real or realistic inputs, each with the ideal output and a list of **hard fails**. `evals/parser.golden.json` is a good example: it covers English, Arabic, Franco-Arabic, typos, prompt injection and impossible requests. Grade with code where possible (exact fields), and with an LLM judge plus a written rubric only for fuzzy parts like tone.

2. **Invariant checks (online, on every request).** Code that checks every output: every stop is open, inside the time window, under the drive limit, not in an avoided category. Count violations; they should be zero.
3. **Product metrics (online).** What users actually do: picked a plan, swapped a stop, shared it, opened a shared link. See the data doc.

Rules:

- **Change one thing, re-run the eval, compare.** Prompts, models and effort levels are all experiments.
- **Keep a held-out slice** you don't tune on, so you don't overfit the prompt to the golden set.
- **Add every production bug to the golden set.**

---

7. Guardrails

- **Prompt injection.** Anything the user or the web wrote (free text, captions, place descriptions, saved-reel data from the browser) is data, not instructions. Limit the damage: the model may only pick from ids we give it, and code checks the result.
- **Cost.** Rate limits per visitor *and* a global daily budget. Alert on spend. Cap retries.
- **Latency.** Set a budget per route (for example, parse < 1.5 s, the whole plan < 5 s) and measure p50 and p95.
- **Privacy.** Log what you need to improve the product, with an anonymous id, and a retention period. No names, phone numbers or precise home locations in logs.

---

8. Checklist for a new AI feature

Copy this into the PR description.

- [ ] Jobs listed, each with its tool (code / search / LLM / agent / ML) and a one-line reason
- [ ] Hard facts and constraints checked in code, not by the model
- [ ] Model and effort chosen **from an eval run**, not by feel; cheaper options tried
- [ ] Output schema + code check after it; every failure type has a user message
- [ ] Latency budget and worst case with retries written down
- [ ] Golden set with hard fails, runnable in CI
- [ ] Logs: one AI-call log line per request + product events (impressions included)
- [ ] Cost: rate limit, retry cap, rough cost per request written down
- [ ] Injection: untrusted text marked as data; the model's output limited to safe choices

Yalla AI review (November 2026)

> The playbook applied to our own code: what is built today, what's wrong with it, and where to take it. Written from an AI tech lead / ML staff point of view. Line numbers refer to the code at the time of writing.

Summary

- The LLM earns its place in **two** jobs: understanding what users type (including Arabic and Franco-Arabic) and writing the copy.
- It is also doing a third job it's bad at: **scheduling the day** (times, opening hours, driving). Our code already contains most of a planner, but today it mostly fixes the model's mistakes after the fact.
- The job the PRD calls critical, **understanding the request**, is mostly English keyword matching in code. Arabic input doesn't steer the place search at all.
- **Nothing is logged**, so we can't evaluate, tune or ever train a recommendation model. Fixing that is the cheapest, most valuable step and should start now (see data doc).
- A collaborative-filtering recommendation engine isn't possible yet. It becomes possible once we log interactions.

---

1. What's built today

```
flowchart LR
  U[User: text, chips, profile] --> K[Keyword matching<br/>areas, cuisines, categories]
  K --> G[Google Places search<br/>up to 18 queries]
  C[Hand-picked Lebanon places] --> F
  S[Saved reels from the browser] --> F
  G --> F[Hard filters<br/>closed, avoided, over budget]
  F --> O[ONE Opus 5.5 call<br/>parse + pick + order + schedule + diversify + write copy]
  O --> R[Code repair<br/>recompute times, swap closed stops, cut overflow]
  R --> I[Firecrawl: Instagram handles]
  I --> P[3 plans on screen]
  P --> SW[Swap / Add buttons<br/>nearest place of same category]
```

Layers

#	Layer	Where	AI?
1	Input check (Zod)	`plans.functions.ts:33`	No
2	Understanding the request: area names found in the text (`:139-141`), a fixed cuisine word list (`live.server.ts:68-94`), category words (`:205-213`)	`plans.functions.ts` , `live.server.ts`	No

#	Layer	Where	AI?
3	Candidates: hand-picked list + Google Places text search per area × category, rating ≥ 4 and ≥ 25 reviews, 6-hour in-memory cache	`places.ts`, `live.server.ts:96-208`	No
4	Hard filters: closed that day, avoided category, over budget	`plans.functions.ts:256-265`	No
5	**Building the plans: one Opus 5.5 call** at medium effort with up to ~180 candidates as JSON	`plans.functions.ts:313-349`, `ai.server.ts`	**LLM**
6	Checking and repairing: Zod, recompute times, replace closed stops, cut overflow	`plans.functions.ts:361-418`	No
7	Extras: Instagram handle via Firecrawl web search	`live.server.ts:211-253`	No
8	Swap / Add: nearest place of the same category, then rating	`plans.tsx:217`, `:490`	No

AI use cases

Use case	Function	Model	What the model does
Build plans	`generatePlans`	Opus 5.5, medium	Everything in layer 5
Read a reel	`readReel` (`media.functions.ts:67`)	Opus 5.5, low	Pulls the place name, area and dish from a TikTok/YouTube caption; Google then finds the place
Status ping	`aiStatus`	Opus 5.5, low	Proves the key works; cached 5 minutes

What's done well:

- All AI calls go through one wrapper (`askJSON`) with structured output, typed failure kinds and server-side fallback.
- The model may only use place ids from the candidate list, so it can't invent a place.
- The code checks opening hours after the model.
- A thoughtful golden eval set already exists.

---

2. Findings, most important first

F1. The LLM schedules the day, and code overrides it

- The code recomputes every arrival time and keeps whichever is later, the model's or its own (`plans.functions.ts:381-382`).
- It swaps out stops the model placed at closed times (`:383-403`).
- It cuts stops past the end time (`:404`).

- The two disagree on drive time. The prompt says about 1.5 minutes per km (`:320`); the code uses `d*1.3/0.7` , about 1.86 minutes per km (`places.ts:460`). The code's plan therefore always runs later than the model's, which causes more swapped and cut stops.

Why it matters: we pay Opus to do arithmetic we then throw away. This is a classic routing problem: pick a few stops, each with opening hours, inside a time and driving budget. With 50–150 candidates and 2–5 stops, a simple beam search finds valid plans in milliseconds, and they are always valid.

F2. Understanding the request is keyword matching

- The PRD plans an LLM step that turns the text into structured fields; the code doesn't have one.
- The Google search is driven by English substring matches. "بدنا نتغدى بالبترون" doesn't match "Batroun", so `areas` stays empty and the search falls back to Beirut and Batroun.
- `evals/parser.golden.json` is a strong eval set (Arabic, Franco-Arabic, prompt injection, impossible requests), but the parser it tests doesn't exist and nothing runs it.

F3. One large model for every route

- Reading a reel is simple name extraction and runs on Opus. Haiku is the natural fit.
- The planner is one Opus call with ~180 candidates as JSON in, 3 plans out. That makes the PRD's 5-second goal unlikely; we haven't measured it, so measure first.
- Worst-case latency is long. The SDK retries once (`ai.server.ts:12`), the loop tries twice (`plans.functions.ts:342`), and each call has a 60-second timeout.

F4. Swap and Add ignore the rules (fixed)

Swap and Add picked the nearest place of the same category from `allPlaces()` . They didn't check opening hours, budget or diet (`plans.tsx:217-220` , `:490-492`). After the planner carefully repaired the plan, one tap could bring back a closed or non-halal place.

Fixed in PR #4. Both now go through `src/lib/alternatives.ts` , which checks opening hours at the arrival time, avoided categories, budget and diet, with unit tests in `tests/alternatives.test.ts` . Live places found by a diet search on Google now carry a `diet` tag, so a vegan user's Swap only offers meals Google returned for "vegan". Still open: swapping one stop moves the arrival times of later stops, and those later stops aren't checked again. The page already warns "Closed at this time. Swap it?" when that happens.

F5. Nothing is logged

- No record of prompts, outputs, tokens, cost, latency or stop reasons.
- No product events: which plan was picked, swaps, removals, shares, opened links.
- The profile and saves live only in the browser's storage.

We can't measure quality, compare a change against the current version, or train anything. Swaps and removals are free "this suggestion was wrong" labels, and today we throw them away.

F6. Server memory is shared across users

- The `LIVE` place registry is shared by every visitor and fully cleared at 3000 places (`places.ts:437`).
- The Google search cache in `live.server.ts:7` is never cleaned up, and its keys include any world city name, so it can grow without limit.
- The rate limiter and caches live per server instance, so on serverless hosting they mostly reset.

F7. Smaller issues

- Sunset times are hard-coded for Lebanon. Abroad, the model is asked to guess "local sunset". A sunset formula would be exact.
- "Prefer high rating with many reviews" is left to the model. Do it in code with a rating adjusted for review count (a Bayesian average), so 4.9 from 30 reviews doesn't beat 4.6 from 3,000.
- Saved-place text from the browser goes into the prompt. The damage is limited, because the model can only pick from the given ids and share links already carry any text anyone wants to put in.

---

3. Target architecture

```
flowchart LR
  U[User input] --> P1[1. Parse<br/>Haiku 5.5, structured output<br/>tested in CI with the golden set]
  P1 -->|area, slots, times, people,<br/>budget, avoid, clarify?| R[2. Retrieve<br/>Google Places per slot<br/>+ hand-picked + saved]
  R --> SC[3. Score in code<br/>adjusted rating, distance, newness,<br/>profile match, saved boost]
  SC --> PL[4. Plan in code<br/>beam search under hours, time window<br/>and drive limits; top 3 that differ]
  PL --> RR{5. Optional taste re-rank<br/>Sonnet 5.5 picks 3 of top 8}
  RR --> W[6. Write copy<br/>Sonnet 5.5: titles, vibe, reasons<br/>for fixed plans]
  W --> OUT[3 plans]
  OUT --> LOG[(7. Log everything:<br/>AI calls + user events)]
  LOG -.later.-> SC
```

Step	Tool	Latency target	Notes
1. Parse	Haiku 5.5	< 1.5 s	Output is the `schema` in `parser.golden.json`. Ask one clarifying question only when both area and activity are missing

Step	Tool	Latency target	Notes
2. Retrieve	Google Places	parallel, cached	Driven by parsed slots, not keyword matching
3. Score	Code	ms	One formula; later replaced by a trained ranking model
4. Plan	Code	ms	Same rule checks reused by Swap and Add (fixes F4)
5. Re-rank	Sonnet 5.5 (optional)	< 2 s	Only if A/B tests show it beats the scoring formula
6. Copy	Sonnet 5.5 or Haiku 5.5	< 2 s, can stream	It can't change places, only words
7. Log	DB / analytics	async	See data doc

What we get: plans that always respect the rules, much lower latency and cost per plan, each step testable on its own, and a feedback loop that lets ranking improve with use. The LLM stays where it adds something only an LLM can: the Arabic and Franco-Arabic input, and the local tone.

---

4. Order of work

Each step is shippable on its own:

1. **Logging** (AI-call log + product events). Nothing else can be measured without it, and the data can't be collected after the fact.
2. ~~**Fix Swap / Add** to use the same hours, budget and diet checks as the planner (F4).~~ Done.
3. **Parser on Haiku** + run `parser.golden.json` in CI; drive the Google search from the parsed slots (F2).
4. **Move reading reels and the status ping to Haiku** (F3).
5. **Score + plan in code**; the LLM writes copy only. A/B test against the current planner on picked-plan and share rates (F1).
6. **Shared cache / rate limit** in a real store (F6).
7. **Ranking model, then recommendations**, once data volume allows (see data doc).

Data and recommendations

> "Should we build a collaborative-filtering recommendation engine?" **Short answer:** not yet, but **start collecting the data today.** A recommendation engine is only as good as the interaction history behind it, and history can't be collected after the fact. Logging is cheap, and it pays off immediately in evals and debugging long before any model is trained.

Read first: Playbook · Yalla AI review

---

1. Why not collaborative filtering today

Collaborative filtering (CF) learns "people like you liked X" from a table of **users × items × interactions**. Today we have:

- **No users.** There are no accounts, and profiles live only in the browser.
- **No interactions stored.** Picks, swaps and shares disappear.
- **Low frequency.** People plan weekends maybe once a week, so each user produces few data points.
- **A new catalogue in each city.** Live Google places appear on demand, so most items have no history.

That is the **cold-start problem** on both sides. CF trained on this would be noise. What works now is **content-based scoring** (what we know about the place and the request) plus logging, with **online learning** to start improving from day one.

---

2. Log #1: AI calls

One row per model call. This is for cost, latency, quality and debugging.

```
type AiCallLog = {
  id: string; // uuid
  requestId: string; // ties together all calls for one plan request
  route: "parse" | "plan" | "copy" | "reel" | "judge" | "status";
  model: string; // "claude-haiku-5-5"
  effort: "low" | "medium" | "high";
  promptVersion: string; // bump on every prompt change, e.g. "parse@3"
  inputTokens: number;
  outputTokens: number;
  cacheReadTokens: number;
  latencyMs: number;
  stopReason: string; // end_turn | max_tokens | refusal | ...
  ok: boolean;
  errorKind?: "credits" | "busy" | "key" | "refused" | "schema" | "other";
  invariantViolations: number; // closed stop, over time, over drive... should be 0
  at: string; // ISO time
};
```

Everything you need is already on the Anthropic response (`usage`, `stop_reason`). `askJSON` in `src/lib/ai.server.ts` is the single place to add it.

3. Log #2: product events

One row per thing the user sees or does. This is the training data for ranking and recommendations.

```
type Event = {
  id: string;
  anonId: string; // random uuid kept in the browser; no personal data
  sessionId: string;
  requestId?: string; // which plan request this belongs to
  type:
    | "plan_requested" // parsed intent, city, chips, time window, group size
    | "plans_shown" // IMPRESSIONS: every place id shown, with its plan and position
    | "plan_opened"
    | "stop_swapped" // from placeId → to placeId  (negative signal for "from")
    | "stop_removed" // negative signal
    | "stop_added" // positive signal, plus which category was missing
    | "plan_shared" // strong positive signal
    | "share_opened" // a friend opened the link: group-level signal
    | "place_clicked" // maps / instagram / website
    | "reel_saved"; // explicit "I want to go here"
  placeId?: string;
  props: Record<string, unknown>; // context: day, time, area, people, budget, prefs
  at: string;
};
```

Rules that save you later:

- **Log impressions, not just clicks.** You can't learn "shown but ignored" if you didn't record what was shown. Without impressions, a ranking model learns popularity, not

preference.
- **Log the context with every event**: day, time window, area, group size, budget, preferences. Recommendations here depend heavily on context; Saturday brunch for 6 isn't Friday dinner for 2.
- **Version everything.** `promptVersion`, `rankerVersion`. Otherwise you can't tell which change helped.
- **Use an anonymous id.** A random id kept in the browser is enough. Accounts can link to it later.
- **Privacy:** update the privacy notice, set a retention period, keep no personal details. Free text from the request is useful for evals; strip anything that looks like a phone number or email.

Where to store it: a single Postgres `events` table is enough for a long time. Lovable Cloud (Supabase) is the path of least resistance in this project. A product analytics tool (e.g. PostHog) adds dashboards and funnels, but keep a raw copy you control, because you'll want to train on it.

---

4. The ladder: from no data to collaborative filtering

Climb one rung at a time. Each rung needs the one before it as a baseline to beat in an A/B test.

Rung	What	Needs	Yalla
0	**Rules + scoring formula**: rating adjusted for review count, distance, opening hours, profile match	Nothing	Do now (target architecture step 3)
1	**Context popularity**: "most shared brunch spots in Batroun on Saturdays"	Event logs, a few weeks	Simple SQL; strong baseline
2	**Online learning with bandits**: each place gets a learned boost; show mostly the best, sometimes explore; reward = kept and not swapped, or shared	Event logs from day one	**The "build it online" option.** Learns from live traffic with very little data
3	**Item-to-item co-occurrence**: "places that appear together in shared plans" / "saved X, also saved Y"	Hundreds of shared plans per city	Powers "more like this reel" and better Add suggestions
4	**Learning to rank**: a gradient-boosted ranking model (e.g. LightGBM LambdaMART) on place + context features, labels from shown / kept / swapped / shared	Thousands of logged sessions with impressions	Replaces the hand-written scoring formula
5	**Collaborative filtering / two-tower models**: user and place embeddings learned from interactions	Many repeat users with many interactions each; accounts	Only when we have a real returning user base

Rung	What	Needs	Yalla
+	**Content embeddings**: embed place descriptions, reviews and reel captions; match on "vibe"	No interaction data needed	Useful early for cold start; pairs well with rungs 3–5

The thresholds are rules of thumb. The real test is always: **does it beat the rung below in an A/B test on share rate?**

"Build it online" vs "start collecting data": both, in this order

1. **This week:** ship the two logs above.
2. **Next:** rung 0 (scoring in code) and rung 1 (context popularity from SQL).
3. **Then:** rung 2, a bandit on top of the scoring formula. This is how to "learn online" with little data. It improves from live traffic, needs no training pipeline, and keeps exploring so new places get a fair chance (cold start).
4. **Later:** rungs 3–5 as volume grows. By then you have months of clean, versioned data with impressions to train on.

An off-the-shelf recommendation service doesn't skip any of this. It needs the same event stream and the same volume.

---

5. Metrics to track from day one

Metric	Meaning	Good direction
Time to first plan (p50 / p95)	Speed	↓ (PRD target: 5 s)
Picked-plan rate	% of requests where a plan is opened	↑
Swap rate per stop	How often a suggestion was wrong	↓
Share rate	The north-star: the plan was good enough to send	↑
Share-open rate	Friends opened it: group reach	↑
Invariant violations	Closed / over-time / over-drive stops shown	= 0
Cost per shared plan	AI + Google spend ÷ shares	↓